Bridging Correctness and Quality in LLM Code Generation: A Hybrid Generate-and-Refine Pipeline

Nouman Munib
Independent Researcher
Gujranwala, Pakistan
noumanmunib27@gmail.com

Abstract

Code generation is important for speeding up bug fixes and preparing new features. However, previous studies still face issues with automatically generated code. Large Language Models (LLMs) increasingly assist developers in code generation, yet research reveals a critical trade-off: some approaches achieve high correctness but produce lower-quality code, while others improve code quality at the cost of correctness. This gap between functional correctness and software quality presents a barrier to industry adoption of LLM-based code generation tools. We propose a hybrid Generate-and-Refine pipeline that addresses this trade-off through a two-stage approach. Stage 1 uses Few-Shot prompting for functionally correct solutions, while Stage 2 implements three candidate refinement strategies: specialized LLM prompting, rule-based automated refactoring, or Retrieval-Augmented Generation. We will empirically evaluate all three refinement approaches using the APPS dataset and quality metrics, determining which strategy best improves code quality while maintaining correctness.

CCS Concepts

• **Software and its engineering → Automatic programming**; **Source code generation**; **Software development methods**; **Maintaining software**; • **Computing methodologies → Natural language processing**.

Keywords

Code Generation, Large Language Models, Code Quality, Automated Refactoring, Few-Shot Prompting

ACM Reference Format:
Nouman Munib. 2026. Bridging Correctness and Quality in LLM Code Generation: A Hybrid Generate-and-Refine Pipeline. In *Proceedings of The 5th International Workshop on Natural Language-Based Software Engineering (NLBSE '26).* ACM, New York, NY, USA, 2 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large Language Models (LLMs) have revolutionized code generation through few-shot prompting, achieving high performance rates on competitive programming benchmarks. However, a fundamental problem persists. Studies report that some LLM strategies (i.e., zero-shot, few-shot, etc.) can generate automatic code that produces functionally correct solutions; however, the code exhibits poor quality characteristics, including unmaintainable structures, inefficient algorithms, and suboptimal patterns. Contrariwise, some strategies improve quality but sacrifice performance, creating an undesirable trade-off. Recent empirical research has systematically documented this correctness-quality tension.

Studies demonstrate that few-shot prompting achieves high correctness (Pass@1 = 0.87) but produces lower-quality code, while fine-tuning sacrifices correctness (Pass@1 = 0.47) to improve quality metrics [1]. When models attempt to improve code quality through refactoring, they frequently introduce bugs and correctness issues [2]. Furthermore, analysis of maintainability improvements shows that while LLMs can enhance certain quality dimensions, these improvements often come at the cost of functional reliability [3].

This work proposes a solution to this gap. We introduce a hybrid Generate-and-Refine pipeline that decouples code generation (prioritizing correctness) from code refinement (optimizing quality). Rather than choosing between correctness and quality, our approach aims to maintain Pass@1 $\geq$ 0.80 (passing all test cases on first attempt) through few-shot generation while improving code quality through targeted refinement strategies. By evaluating three distinct refinement approaches, we seek to identify which strategy most effectively bridges correctness and quality without compromising either criterion.

2 Proposed Approach

Our hybrid pipeline, as illustrated in Figure 1, consists of two specialized stages:

Stage 1: Code Generation for Correctness. We will employ Few-Shot prompting with state-of-the-art LLMs [5] to generate initial code solutions, prioritizing functional correctness by leveraging the model's ability to generate working solutions with minimal examples.

Stage 2: Code Refinement for Quality. We will evaluate three candidate refinement strategies applied to correct code, each targeting different quality dimensions: semantic understanding through LLM analysis, deterministic rule-based transformations, and exemplar-guided refinement through retrieval.

(1) **LLM Refiner:** Specialized LLM prompting to analyze correct code and recommend quality improvements while maintaining correctness. This approach leverages semantic understanding for holistic refactoring without losing functional guarantees.

NLBSE '26, Rio de Janeiro, Brazil

ACM ISBN 978-1-4503-XXXX-X/2026/04
https://doi.org/XXXXXXX.XXXXXXX

(2) **Rule-Based Refiner:** Automated refactoring rules from industry best practices (e.g., method extraction, variable renaming, optimization) applied deterministically for reproducible, explainable transformations. This strategy ensures consistency and interpretability.
(3) **RAG Refiner:** Retrieval-Augmented Generation [6] that retrieves exemplary code snippets from curated repositories of high-quality implementations to guide refinement through context-aware prompting. This approach grounds refinement in real-world quality exemplars.

Each refinement strategy will be evaluated separately to determine which approach most effectively improves code quality without compromising correctness. Intermediate validation will ensure correctness is preserved after refinement by re-executing all test cases after each refinement step, maintaining our threshold before proceeding to quality assessment. In cases where refinement reduces correctness below our threshold, the pipeline will discard the refined version and retain the original few-shot-generated code.

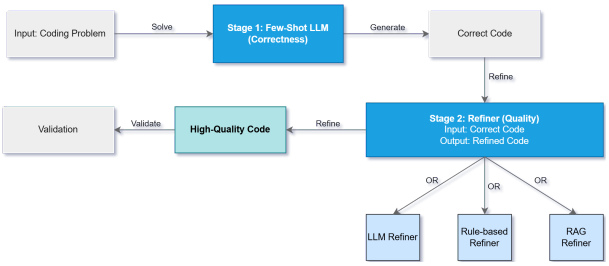

Figure 1: Two-stage Hybrid Generate-and-Refine Pipeline showing input problem, Few-Shot LLM generation, correctness validation, and three candidate refinement approaches (LLM, Rule-Based, RAG) leading to high-quality code.

3 Evaluation and Expected Contributions

We will evaluate the pipeline using the APPS dataset [4] (480 programming problems across three difficulty levels, selected to provide diverse complexity while remaining computationally feasible). We will measure correctness (Pass@1 ≥ 0.80) and quality metrics (SonarQube: bugs, code smells, maintainability index) compared against human-written code baselines. Our evaluation focuses on Python-based competitive programming tasks, leaving generalization to other languages and domains as future investigation.

Expected contributions include:

(1) **Empirical Comparative Analysis.** A comprehensive evaluation of three refinement strategies for improving LLM-generated code quality while preserving the high correctness standards that few-shot prompting achieves.
(2) **Practical Guidance.** Insights into the most effective refinement approach for balancing correctness and quality in production code generation pipelines, informing practitioners on real-world deployments.
(3) **Modular Framework.** A flexible architecture enabling easy substitution of refinement strategies and adaptation to different quality metrics or programming domains.

4 Future Work

While the automated metrics-based evaluation provides quantitative insights, a critical question remains: *Do these measurements align with how expert developers actually perceive code quality?* As a next step, we plan a developer-centric evaluation where experienced practitioners evaluate code snippets using carefully designed dimensions, including: (i) comprehension and intent, (ii) debugging confidence, (iii) change resilience, (iv) idiomatic quality, and (v) production readiness. This study will test whether automated improvements correlate with genuine human-perceived quality or whether key quality dimensions elude automated tools.

Results from this developer-centric evaluation could significantly strengthen the work and may be incorporated into an expanded ICSE 2026 paper or submitted as a follow-up publication. We also intend to expand our evaluation across multiple programming languages and assess integration with real-world development workflows.

5 Conclusion

This work will address the correctness-quality trade-off identified in recent studies [1] by proposing and planning to empirically evaluate three refinement strategies within a hybrid generate-and-refine framework. By aiming to maintain high functional correctness (Pass@1 ≥ 0.80) while improving measurable code quality, we seek to provide practitioners with actionable guidance for deploying LLM-based code generation in production environments.

Acknowledgments

I acknowledge the use of AI-assisted tools for writing clarity and structural refinement. All research contributions, methodology, and intellectual content are my own.

References

[1] Molison, A. S., Moraes, M., Melo, G., et al. 2025. Is LLM-Generated Code More Maintainable & Reliable Than Human-Written Code? *arXiv preprint arXiv:2508.00700v1 [cs.SE]*.
[2] Jiang, N., Lutellier, T., and Tan, S. H. 2024. What's Wrong With Your Code Generated By Large Language Models? In *Proceedings of the International Conference on Software Engineering (ICSE)*.
[3] Figueiredo, R., Alves, V., and Oliveira, S. 2025. Evaluating The Effectiveness Of LLMs In Fixing Maintainability Issues. In *Proceedings of the International Conference on Software Analysis, Evolution and Reengineering (SANER)*.
[4] Hendrycks, D., Basart, S., Kadavath, S., Mazeika, M., Arora, A., Guo, E., Burns, C., Puranik, S., He, H., Song, D., and Steinhardt, J. 2021. Measuring Coding Challenge Competence With APPS. In *Advances in Neural Information Processing Systems (NeurIPS)*, 7566–7583.
[5] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. 2020. Language Models Are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1877–1901.
[6] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., et al. 2020. Retrieval-Augmented Generation For Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 9459–9474.
[7] SonarSource. 2024. SonarQube: Code Quality And Security. Retrieved October 27, 2025, from https://www.sonarqube.org/